Developing a ReAct Sudoku Agent

CS 4300-01 Artificial Intelligence | Curtis Larsen

Git repository

https://github.com/sshb4/ReActSudokuAgent

REVIEW COPY - student code review confirmation pending

System and component interactions

A local language model selects one typed tool request at a time. The application owns the board, knowledge base, action budget, trace, and terminal status. Only an independently validated submission can establish success.

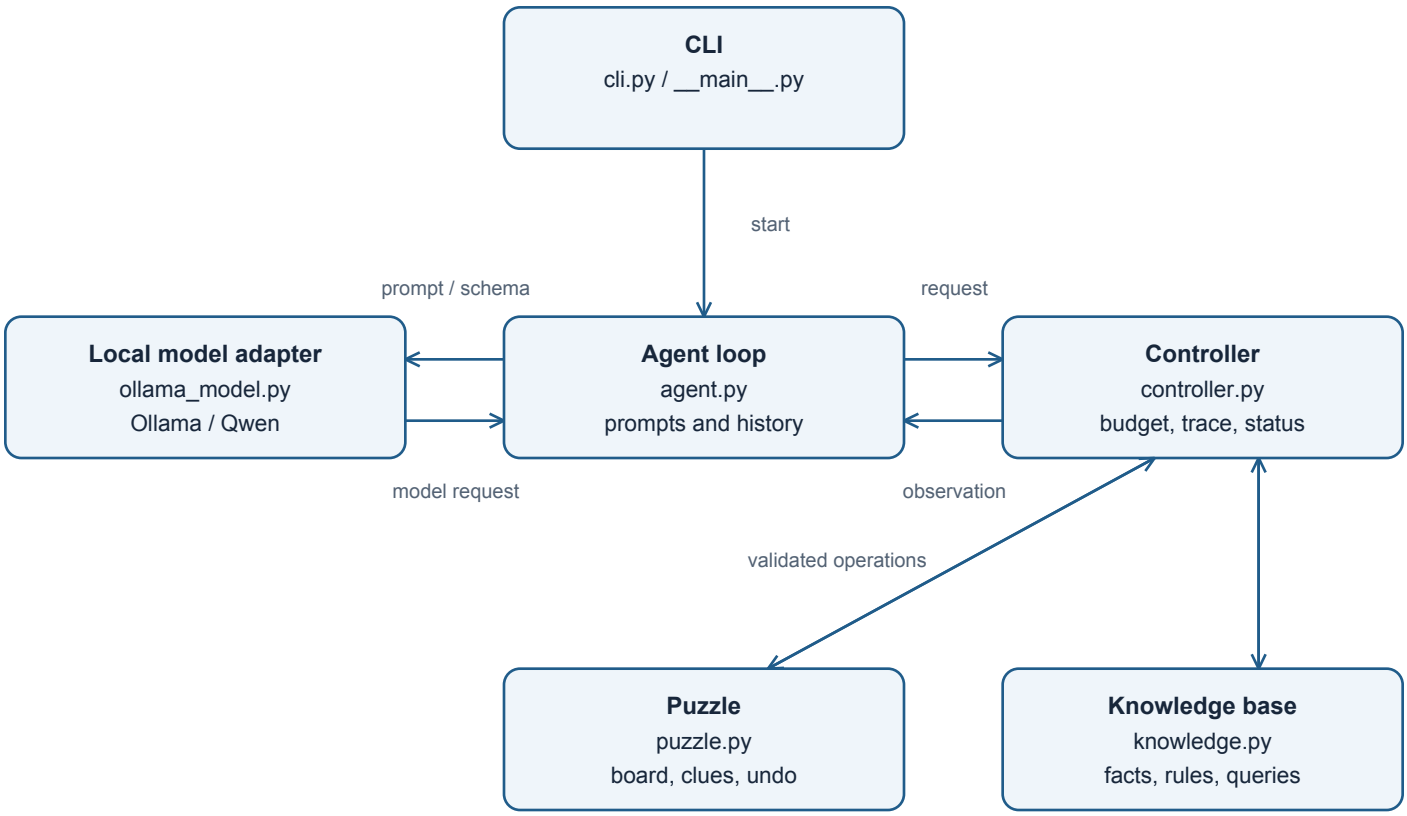


Figure 1. Requests go through the controller; observations return to the model. schemas.py defines the shared typed interfaces, and model.py defines the runtime-independent model protocol.

State boundaries

Clues cannot be edited. Rejected operations leave puzzle and KB state unchanged. Every attempted tool request consumes an action. Undo removes the latest accepted placement. A successful submission on the final allowed action takes precedence over budget exhaustion.

Development process and evidence

Specifications before implementation

The architecture and public behavior contracts were written first. They define tool schemas, puzzle representation, classical propositional semantics, undo behavior, controller policies, and ordered terminal checks. Main specifications, tests, and implementation were committed in that order; the development log records later revisions honestly.

Tests and coding-agent implementation

Black-box tests were derived from the contracts before the corresponding main function bodies existed. A coding agent implemented a significant portion of the project against those specifications and tests. Later review added an independent truth-table oracle and explicitly labeled white-box fault injection. The complete suite passes 170 cases.

What real-model testing changed

The first small-model run repeatedly targeted a clue and reached its action limit. The controller protected the clues. Board observations were then revised to label empty-cell coordinates and report legal candidates. An unsupported regex feature was replaced, and explicit terminal reasons were added to tool observations. Tests were written for these gaps before the corresponding fixes.

Verification and limits

Ollama 0.40.6 with Qwen 2.5 1.5B completed the three-empty-cell example in 18 model-selected tool requests. The saved live trace includes fact and rule updates, an entailment query, placements, and validated submission. This demonstrates one near-complete puzzle; harder-puzzle reliability is not established.

Student review

A concrete code-review walkthrough is included in docs/student-review.md. Student confirmation is still pending; this review copy does not claim that the student's review responsibility has already been completed.

Takeaway I plan to keep

I plan to decide and write down what each function should do before coding it, so I have a clear contract to implement and test.